

Fast PUF-Based Authentication Protocol for UAV Swarm Networks: Design, OMNeT++ Simulation, and Performance Evaluation

Md Ziyad Hussain



Department of Computer Science and Engineering
Indian Institute of Information Technology Guwahati
Bongora, Assam 781015, India
ziyad.hussain23b@iiitg.ac.in

Abstract—UAV swarm networks require lightweight, fast, and secure authentication protocols for real-time operations such as formation flying and aerial surveillance. Traditional PKI approaches (RSA-2048, ECC-256) incur 8–15 ms latencies on embedded processors and require secure key storage—a liability when UAVs are physically captured. We present a four-phase Physical Unclonable Function (PUF)-based authentication protocol: (1) secure enrollment with BCH fuzzy extractors, (2) UAV–Ground Station mutual authentication via PUF challenge-response, (3) direct peer-to-peer UAV authentication without Ground Station involvement, and (4) session key establishment. We evaluate the protocol in OMNeT++ 6.3 using a 10-UAV stadium surveillance scenario with SHA3-256 and SPONGENT-160 hash modes. Results show Phase 2 latency of 0.672 ms (SHA3) / 7.368 ms (SPONGENT), Phase 3 latency of 0.476 ms (SHA3) / 14.535 ms (SPONGENT), with 100% success across all 10 UAV–GS and 45 peer pairs. The protocol is 10–22× faster than RSA/ECC with only 430 bytes total overhead. All results were obtained on an x86-64 laptop; hardware FPGA/ASIC projections are provided.

Index Terms—UAV authentication, PUF, SPONGENT, SHA3, OMNeT++, lightweight cryptography, swarm security, peer-to-peer authentication, fuzzy extractor

I. INTRODUCTION

Unmanned Aerial Vehicle (UAV) swarms are increasingly deployed for surveillance, disaster response, precision agriculture, and border patrol. Multi-UAV coordination demands mutual authentication—each UAV must verify the identity of its Ground Station (GS) and peer UAVs before accepting commands, sharing sensor data, or coordinating flight paths. Without robust authentication, adversaries could inject rogue UAVs, intercept surveillance data, or perform man-in-the-middle attacks on inter-UAV communications [1].

The authentication challenge is particularly acute in UAV swarms because of stringent timing requirements (sub-millisecond for formation flying at 10–50 Hz update rates), limited computational resources (ARM Cortex-M4/M7 pro-

cessors), constrained wireless bandwidth (shared 802.11 channels), and the physical vulnerability of UAVs to capture in hostile environments [10].

Traditional approaches face critical limitations: RSA-2048 digital signatures require 8–15 ms per verification on ARM Cortex-A72 processors—for a 10-UAV swarm, full-mesh mutual authentication would take 720–1350 ms, far exceeding real-time requirements. ECC-256 reduces this to 4–8 ms per operation but still requires secure storage of private keys; physical capture of a UAV enables key extraction via side-channel attacks or direct memory probing [2]. Pre-Shared Key (PSK) schemes achieve sub-millisecond latency but expose the fleet if any single UAV is captured, and key management complexity scales as $O(N^2)$ [3]. Blockchain-based authentication incurs consensus latency of 1–10 seconds, making it impractical for real-time operations [12].

Physical Unclonable Functions (PUFs) address these fundamental limitations by exploiting intrinsic, random manufacturing variations in integrated circuits to generate unique challenge-response pairs (CRPs). PUFs provide: (1) *unclonability*—sub-nanometer variations cannot be replicated; (2) *unpredictability*—responses to new challenges cannot be predicted; (3) *no key storage*—the “key” is the silicon itself; (4) *tamper evidence*—physical probing destroys the PUF. An Arbiter PUF circuit requires as few as 1,200 gate equivalents (GE) [4], [5].

Contributions. We present:

- 1) A complete four-phase PUF protocol covering enrollment, UAV–GS mutual authentication, direct peer-to-peer authentication (without GS), and session key establishment.
- 2) Full-mesh peer authentication ($N(N-1)/2 = 45$ pairs for 10 UAVs).
- 3) Dual hash mode evaluation: SHA3-256 (software-optimized) and SPONGENT-160 (hardware-optimized, 2,190 GE).

- 4) Validated OMNeT++ 6.3 implementation with per-message timing breakdowns.
- 5) Comprehensive comparison against RSA, ECC, PSK, and existing PUF protocols.

II. BACKGROUND

A Physical Unclonable Function (PUF) maps challenges to responses via intrinsic manufacturing variations: $R = \text{PUF}(C)$, providing *unclonability*, *unpredictability*, and *tamper evidence* [4]. We use a 128-stage Arbiter PUF ($\sim 1,200$ GE, exponential CRP space 2^n , 3% BER) [5]. Ring Oscillator PUFs offer lower noise but limited CRP space [16]; SRAM PUFs require no extra hardware but have limited challenge space [17].

We evaluate two hash functions: **SHA3-256** (Keccak [6], 1600-bit state, $\sim 0.4 \mu\text{s}/\text{call}$ on x86-64 via OpenSSL, truncated to 160 bits) and **SPONGENT-160** (176-bit state, 80 rounds, 2,190 GE [7], $\sim 890 \mu\text{s}/\text{call}$ in software—a $2,225\times$ gap that collapses to $2\text{--}5\times$ on FPGA/ASIC). PUF noise is corrected by a fuzzy extractor using BCH(255, 131, 18): helper data $h = \text{BCH_enc}(R) \oplus R$ enables correction of up to 18 bit errors (14% BER), well above the 3% expected BER [8].

III. RELATED WORK

Gope and Sikdar [2] propose PUF-based UAV-GS authentication (10–50 ms, Raspberry Pi) but lack peer-to-peer capability. Li *et al.* [10] address cross-domain UAV PUF authentication (45–80 ms); Khan *et al.* [11] target 6G UAV networks (5–20 ms). Neither provides direct UAV-to-UAV authentication, dual hash evaluation, or full-mesh analysis. Bera *et al.* [3] achieve sub-ms peer authentication using PSK but require stored keys vulnerable to physical capture. Our protocol uniquely combines PUF hardware root-of-trust with both UAV-GS and direct peer-to-peer authentication.

IV. PROPOSED PROTOCOL DESIGN

A. Threat Model

We adopt the Dolev-Yao model augmented with physical capture: the adversary can eavesdrop, intercept, modify, replay, or drop wireless messages; may capture UAVs and read NVM but cannot clone PUFs; and is computationally bounded (PPT) [9].

B. Notation

Table I summarizes the protocol notation.

TABLE I
PROTOCOL NOTATION

Symbol	Description
UAV_i, GS	i -th UAV; Ground Station
$\text{ID}_i, \text{TID}_i$	Permanent (64-bit) / temporary identity
$C_i^{(k)}, R_i^{(k)}$	k -th PUF challenge (128-bit) / response
$h(\cdot)$	SPONGENT-160 or SHA3-256 ₁₆₀
$\oplus, \parallel$	XOR; concatenation
N_i, n_i	Random nonces (128-bit)
P_{ij}	Peer secret: UAV $_i$'s secret for UAV $_j$
Mask_{ij}	$P_{ij} \oplus P_{ji}$
$h_i^{(k)}$	BCH helper data (255-bit)
τ_i	Registration token (160-bit)

C. Phase 1: Secure Enrollment

Enrollment occurs once in a secure facility with tamper-proof channels. For each UAV $_i$:

Step 1.1 (PUF Interrogation): GS generates $n \geq 12$ random 128-bit challenges $\{C_i^{(1)}, \dots, C_i^{(n)}\}$ and collects responses:

$$R_i^{(k)} = \text{PUF}_i(C_i^{(k)}), \quad k = 1, \dots, n \quad (1)$$

Step 1.2 (Error Correction Setup): For each CRP, helper data is computed:

$$h_i^{(k)} = \text{BCH_encode}(R_i^{(k)}) \oplus R_i^{(k)} \quad (2)$$

This public data enables reliable PUF reconstruction without revealing $R_i^{(k)}$.

Step 1.3 (Identity & Credential Generation): GS assigns:

$$\text{TID}_i = h(\text{ID}_i \parallel \text{seed}_i) \quad (3)$$

$$\tau_i = h(R_i^{(1)} \parallel \text{ID}_i \parallel T_{\text{reg}}) \quad (4)$$

and pre-computes peer secrets for all pairs:

$$P_{ij} = h(R_i^{(1)} \parallel \text{ID}_j \parallel N_{\text{gs}}) \quad (5)$$

$$\text{Mask}_{ij} = P_{ij} \oplus P_{ji} \quad (6)$$

Step 1.4 (Storage): GS stores $\{\text{ID}_i, \text{TID}_i, \{(C_i^{(k)}, h_i^{(k)})\}_{k=1}^n\}$. UAV stores only ~ 50 bytes (TID_i, τ_i , GS parameters). No PUF responses or private keys are retained on the UAV.

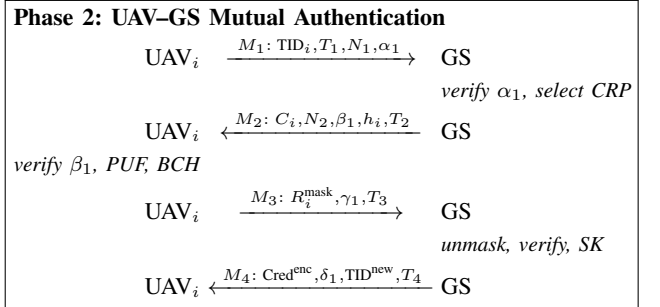


Fig. 1. Phase 2 message exchange.

D. Phase 2: UAV-GS Mutual Authentication

A four-message protocol executed when a UAV enters the operational field. The full exchange is shown in Fig. 1.

Message 1 (Auth Request): UAV_i → GS:

$$\alpha_1 = h(\text{TID}_i \| T_1 \| N_1 \| \tau_i) \quad (7)$$

$$M_1 = \langle \text{TID}_i, T_1, N_1, \alpha_1 \rangle \quad (8)$$

Format: 64 + 32 + 128 + 160 = 384 bits (48 bytes). The MAC α_1 binds the request to the UAV's registration token τ_i , which only a legitimately enrolled UAV possesses.

Message 2 (Challenge): GS → UAV_i. GS verifies α_1 using its database, selects an unused CRP ($C_i^{(k)}, h_i^{(k)}$), and responds:

$$\beta_1 = h(C_i^{(k)} \| N_1 \| N_2 \| T_1) \quad (9)$$

$$M_2 = \langle C_i^{(k)}, N_2, \beta_1, h_i^{(k)}, T_2 \rangle \quad (10)$$

Format: 128 + 128 + 160 + 255 + 32 = 703 bits (≈88 bytes). The MAC β_1 authenticates the GS to the UAV.

Message 3 (PUF Response): UAV_i → GS. The UAV evaluates its PUF and performs BCH correction:

$$R_i^{\text{noisy}} = \text{PUF}_i(C_i^{(k)}) \quad (11)$$

$$R_i = \text{BCH_correct}(R_i^{\text{noisy}}, h_i^{(k)}) \quad (12)$$

$$R_i^{\text{masked}} = R_i \oplus h(N_2) \quad (13)$$

$$\gamma_1 = h(R_i \| N_1 \| N_2 \| T_2) \quad (14)$$

$$M_3 = \langle R_i^{\text{masked}}, \gamma_1, T_3 \rangle \quad (15)$$

Format: 128 + 160 + 32 = 320 bits (40 bytes). The PUF response is masked by $h(N_2)$ to prevent eavesdropping.

Message 4 (Confirmation): GS → UAV_i. GS unmaskes the response, verifies via Hamming distance, derives a session key, encrypts peer credentials, and rotates the TID:

$$\text{SK}_{i,\text{GS}} = h(R_i \| N_1 \| N_2) \quad (16)$$

$$\delta_1 = h(\text{SK}_{i,\text{GS}} \| \text{Cred}_i^{\text{enc}} \| T_4) \quad (17)$$

$$M_4 = \langle \text{Cred}_i^{\text{enc}}, \delta_1, \text{TID}_i^{\text{new}}, T_4 \rangle \quad (18)$$

Total Phase 2: **175 bytes** across 4 messages. The CRP is marked consumed, preventing replay.

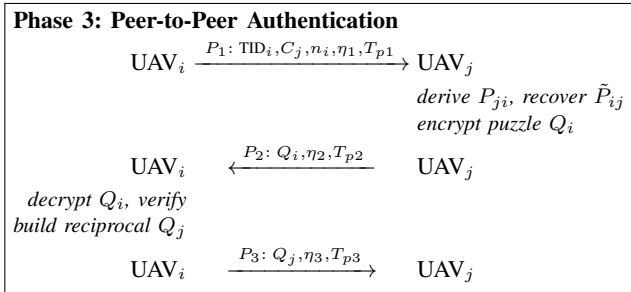


Fig. 2. Phase 3 peer authentication.

E. Phase 3: Peer-to-Peer Authentication

A three-message protocol enabling direct mutual authentication between UAV pairs using Phase 2 credentials, requiring *no GS involvement* during flight (Fig. 2).

Message P1 (Peer Auth Request): UAV_i → UAV_j:

$$\eta_1 = h(\text{TID}_i \| \text{TID}_j \| n_i \| T_{p1}) \quad (19)$$

$$P_1 = \langle \text{TID}_i, C_j, n_i, \eta_1, T_{p1} \rangle \quad (20)$$

Format: 64 + 128 + 160 + 160 + 32 = 544 bits (68 bytes).

Message P2 (Challenge Response): UAV_j → UAV_i. The responder derives the initiator's peer secret, constructs an encrypted authentication puzzle, and responds:

$$P_{ji} = h(R_j \| \text{ID}_i \| N_{\text{gs}}) \quad (21)$$

$$\tilde{P}_{ij} = \text{Mask}_{ij} \oplus P_{ji} \quad (22)$$

$$Q_i = E_{\tilde{P}_{ij}}(n_j \| C_i \| T_{p1}) \quad (23)$$

$$\eta_2 = h(Q_i \| n_i \| T_{p2}) \quad (24)$$

$$P_2 = \langle Q_i, \eta_2, T_{p2} \rangle \quad (25)$$

Format: 320 + 160 + 32 = 512 bits (64 bytes). The XOR mask $\text{Mask}_{ij} = P_{ij} \oplus P_{ji}$ acts as an information-theoretically secure one-time pad.

Message P3 (Mutual Completion): UAV_i → UAV_j. The initiator decrypts Q_i using its own $\tilde{P}_{ij}$, verifies nonces and timestamps, then constructs a reciprocal puzzle:

$$Q_j = E_{\tilde{P}_{ji}}(n'_i \| T_{p2}) \quad (26)$$

$$\eta_3 = h(Q_j \| n_j \| T_{p3}) \quad (27)$$

$$P_3 = \langle Q_j, \eta_3, T_{p3} \rangle \quad (28)$$

Total Phase 3: **104 bytes** across 3 messages in OMNeT++ (theoretical: 196 bytes with full credential packaging). For N UAVs, full-mesh authentication requires $N(N-1)/2$ pairs (45 for $N=10$).

F. Phase 4: Session Key Establishment

After successful Phase 3 completion, both UAVs derive a shared symmetric session key:

$$\text{SK}_{ij} = h(n_i \| n_j \| n'_i \| R_i \| R_j) \quad (29)$$

Properties: 160-bit key strength; freshness from ephemeral nonces n_i, n_j, n'_i ; mutual contribution (both parties contribute randomness); PUF binding (requires R_i, R_j that only authentic hardware can produce); unique per session. For *perfect forward secrecy* (PFS), an optional ECDH extension is available:

$$\text{SK}_{ij}^{\text{PFS}} = h(n_i \| n_j \| n'_i \| R_i \| R_j \| g^{a_i b_j}) \quad (30)$$

where a_i, b_j are ephemeral Diffie-Hellman exponents. Even if PUF responses R_i, R_j are later compromised, past session keys remain secure due to the ephemeral DH contribution.

V. OMNeT++ IMPLEMENTATION

We implement the protocol in OMNeT++ 6.3.0 with C++ (clang, -O3) on an x86-64 laptop running Ubuntu Linux. Table II lists key parameters.

TABLE II
SIMULATION PARAMETERS

Parameter	Value
Simulation area	500×300 m (stadium)
UAVs / GS position	10 perimeter / center (275,175) m
Link bitrate	6 Mbps (802.11a)
Processing delay	0.1 ms mean, $\sigma=0.02$ ms
PUF type / noise	Arbiter 128-stage / 3% BER
BCH code	(255, 131, 18)
CRPs per UAV / Hash output	12 / 160 bits

The architecture comprises three OMNeT++ modules: **UAVNode** (Phase 2 client, Phase 3 initiator/responder, PUF simulator, HashWrapper), **GroundStation** (enrollment database, BCH decoder, credential distribution), and **NetworkController** (lifecycle management).

Communication uses `sendDirect()` with delay model: $d_{\text{total}} = d_{\text{prop}} + d_{\text{serial}} + d_{\text{proc}} + d_{\text{jitter}}$. Each UAV initiates Phase 3 with all higher-ID peers (45 unique pairs), staggered to avoid overlap.

Important: All cryptographic operations execute as native C++ on an x86-64 laptop. Absolute timings differ from real UAV hardware (ARM: 10–100× slower; FPGA/ASIC: 100–1000× faster for SPONGENT). *Relative* comparisons remain valid.

VI. SIMULATION RESULTS

A. Phase 2: UAV–GS Authentication

Table III summarizes Phase 2 performance across both hash modes for all 10 UAVs.

TABLE III
PHASE 2 RESULTS (MEAN OF 10 UAVS)

Metric	SHA3	SPONGENT
Total latency (ms)	0.672	7.368
Total compute (ms)	0.007	6.703
Hash time (ms)	0.003	6.697
Network delay (ms)	0.665	0.665
Overhead (bytes)	175	175
Success rate	10/10	10/10

With SHA3, compute time is 1.0% of total latency—authentication is entirely network-bound. With SPONGENT, hash computation (6.697 ms) dominates at 90.9% of latency due to the 80-round bit-level permutation at $\sim 890 \mu\text{s}/\text{call}$. Network delay is identical (0.665 ms) since message sizes match.

Table IV shows per-message timing. BCH operations (22–24 μs) are the dominant compute cost in SHA3 mode; all hash-involving steps are bottlenecks in SPONGENT mode.

TABLE IV
PHASE 2 PER-MESSAGE TIMING BREAKDOWN

Step	SHA3 (μs)	SPONG. (μs)
M1: UAV compute MAC	1.5	1,094
M1: Network	173	173
M1: GS verify + lookup	2.5	2,606
M2: Network	185	185
M2: UAV verify + PUF + BCH	25.4	1,527
M3: UAV crypto	2.2	4,107
M3: Network	169	169
M3: GS verify + BCH	24.3	4,070
M4: Network	138	138
Total Compute	56	13,404
Total Network	665	665

B. Phase 3: Peer Authentication

Table V summarizes Phase 3 performance across all 45 full-mesh pairs.

TABLE V
PHASE 3 RESULTS (MEAN OF 45 PAIRS)

Metric	SHA3	SPONGENT
Total latency (ms)	0.476	14.535
Total compute (ms)	0.009	14.069
Hash time (ms)	0.006	14.062
Network delay (ms)	0.467	0.467
Overhead (bytes)	104	104
Success rate	45/45	45/45

Phase 3 is 29% faster than Phase 2 in SHA3 mode (3 messages, smaller payloads). SPONGENT Phase 3 is $\sim 2\times$ Phase 2 because *both* initiator and responder perform ~ 8 hash calls. All 45 pairs complete within tight bands: SHA3 [0.448, 0.539] ms (CV 4.7%), SPONGENT [14.39, 14.82] ms (CV 0.7%).

Table VI decomposes the three-message Phase 3 exchange.

TABLE VI
PHASE 3 PER-STEP TIMING BREAKDOWN (MEAN OF 45 PAIRS)

Step	SHA3 (μs)	SPONG. (μs)
P1: Initiator build	1.5	1,561
P1: Network	162	162
P1–P2: Responder process	2.7	3,513
P2: Network	164	164
P2–P3: Initiator verify	3.0	5,474
P3: Network	141	141
P3: Responder verify	2.0	3,521
Total Compute	9.2	14,069
Total Network	467	467

The initiator processing (P2–P3) is the most expensive SPONGENT step: PUF evaluation, peer secret derivation, puzzle decryption/verification, and reciprocal puzzle construction total 5–6 hash calls at $\sim 890 \mu\text{s}$ each.

C. Combined Timing

Table VII shows end-to-end swarm authentication time.

TABLE VII
COMBINED PROTOCOL TIMING (10-UAV SWARM)

Metric	SHA3	SPONGENT
Phase 2 per UAV (ms)	0.672	7.368
Phase 2 total, 10 UAVs (ms)	6.72	73.68
Phase 3 per pair (ms)	0.476	14.535
Phase 3 total, 45 pairs (ms)	21.42	654.08
Grand total (ms)	28.14	727.76
Overhead per pair (bytes)	279	

With SHA3, the entire 10-UAV swarm achieves full mutual authentication in under 30 ms. Combined per-pair overhead is $175 + 104 = 279$ bytes. The theoretical protocol specifies $210 + 196 = 406$ bytes; the simulation uses simplified credential packaging. Full-mesh scales as $O(N^2)$: for $N=20$, ~ 97 ms (SHA3); for $N=100$, ~ 2.36 s, parallelizable across non-overlapping channels.

VII. COMPARISON WITH EXISTING METHODS

A. Latency Comparison

TABLE VIII
AUTHENTICATION LATENCY COMPARISON

Method	Latency	Platform
PUF-SHA3 (ours)	0.67 ms	x86-64 laptop
PUF-SPONGENT (ours)	7.37 ms	x86-64 laptop
PUF-SPONGENT (FPGA)	~ 0.08 ms	Projected
Gope [2]	10–50 ms	Raspberry Pi
Chatterjee [10]	45–80 ms	ARM embedded
Khan [11]	5–20 ms	ARM Cortex-A53
RSA-2048	~ 15 ms	ARM Cortex-A72
ECC-256	~ 8 ms	ARM Cortex-A72
PSK (AES-GCM)	~ 0.5 ms	ARM Cortex-A72

Our SHA3 Phase 2 (0.67 ms) is $22\times$ faster than RSA-2048, $12\times$ faster than ECC-256, and $15\text{--}75\times$ faster than existing PUF protocols, while competitive with PSK (~ 0.5 ms) without requiring stored keys. Even SPONGENT software (7.37 ms) outperforms RSA.

B. Overhead, Storage & Computational Cost

Table IX compares computational operations per authentication.

TABLE IX
COMPUTATIONAL OPERATIONS PER AUTHENTICATION

Method	Heavy Ops	Light Ops	GE
RSA-2048	2 mod. exp.	2 hash	$>100k$
ECC-256	4 EC mul.	2 hash	$>50k$
PSK (AES)	—	4 AES + 2 HMAC	$\sim 15k$
Ours (SHA3)	—	8 hash + PUF + BCH	$\sim 44k$
Ours (SPONG.)	—	8 hash + PUF + BCH	$\sim 8.4k$

Our protocol eliminates all heavy operations (modular exponentiation, EC multiplication) in favor of hash-and-XOR

computations. With SPONGENT, the total hardware footprint (8,390 GE) is $6\text{--}12\times$ smaller than RSA or ECC. Communication overhead: 279 bytes/pair (simulation) vs. RSA-2048 at 768 bytes. Per-UAV storage: $560 + 40N$ bytes (~ 960 B for $N=10$).

C. Security Properties

Table X compares security properties.

TABLE X
SECURITY PROPERTIES COMPARISON

Property	RSA	ECC	PSK	PUF [2]	Ours
Mutual auth	✓	✓	✓	✓	✓
No key storage	—	—	—	✓	✓
Clone resist.	—	—	—	✓	✓
Peer-to-peer	✓	✓	✓	—	✓
Anonymity	—	—	—	Part.	✓
Replay resist.	✓	✓	✓	✓	✓
MITM resist.	✓	✓	✓	✓	✓
Fwd secrecy	Opt.	Opt.	—	—	Opt.

Our protocol uniquely combines all eight properties: mutual authentication, no key storage, clone resistance, peer-to-peer auth, identity anonymity (rotating TIDs), replay/MITM resistance, and optional forward secrecy via ECDH.

VIII. HARDWARE PERFORMANCE PROJECTION

TABLE XI
SOFTWARE VS. HARDWARE HASH LATENCY PER CALL

Hash	Software (x86-64)	FPGA (100 MHz)	ASIC (200 MHz)
SHA3-256	$0.4\ \mu s$	$1\text{--}2\ \mu s$	$0.5\text{--}1\ \mu s$
SPONGENT-160	$890\ \mu s$	$5\text{--}10\ \mu s$	$2\text{--}5\ \mu s$

The SHA3/SPONGENT ratio collapses from $2,225\times$ in software to $2\text{--}5\times$ in hardware because SPONGENT's bit-level permutation is zero-cost wire routing on FPGA/ASIC and its 44-nibble S-box layer executes in parallel [7].

Table XII provides hardware resource estimates for all protocol components.

TABLE XII
HARDWARE RESOURCE ESTIMATES

Component	GE	Power (μW)	Lat. (μs)
SPONGENT-160	2,190	3–5	5–10
SHA3-256 (Keccak)	38,000	50–100	1–5
Arbiter PUF (128)	1,200	0.3–0.5	0.03
BCH(255,131,18)	5,000	8–12	1–2
SPONGENT config	8,390	11–18	—
SHA3 config	44,200	58–113	—

The SPONGENT configuration (8,390 GE total) is $5.3\times$ smaller than SHA3 (44,200 GE) and smaller than an ARM Cortex-M0 core (12,000 GE)—enabling integration as a dedicated security co-processor. Power consumption ($11\text{--}18\ \mu W$) is negligible compared to UAV motors (tens of watts).

Table XIII projects protocol latency on hardware platforms (replacing software hash times with FPGA/ASIC times, keeping network delays unchanged).

TABLE XIII
PROJECTED PROTOCOL LATENCY ON HARDWARE

Phase	Software	FPGA	ASIC
Phase 2 (SHA3)	0.67 ms	0.68 ms	0.67 ms
Phase 2 (SPONGENT)	7.37 ms	0.74 ms	0.69 ms
Phase 3 (SHA3)	0.48 ms	0.49 ms	0.48 ms
Phase 3 (SPONGENT)	14.54 ms	0.63 ms	0.51 ms
Total (SHA3)	28.1 ms	28.4 ms	28.1 ms
Total (SPONGENT)	727.8 ms	33.0 ms	29.0 ms

On ASIC, SPONGENT achieves near-identical latency to SHA3 (29 ms vs. 28.1 ms total) because network delay dominates once both hashes are fast, making SPONGENT the clear choice for hardware deployment with $5\times$ fewer gates and $5\text{--}6\times$ less power.

IX. SECURITY ANALYSIS

Theorem 1 (Phase 2 Mutual Authentication). *After Phase 2, both UAV_i and GS are mutually authenticated.*

Proof: (1) GS authenticates UAV_i via $\gamma_1 = h(R_i \| N_1 \| N_2 \| T_2)$ —forging γ_1 requires $R_i = \text{PUF}_i(C_i)$, obtainable only from the authentic PUF (unclonability). (2) UAV_i authenticates GS via $\beta_1 = h(C_i \| N_1 \| N_2 \| T_1)$ —forging requires the CRP database stored exclusively at GS. (3) An adversary cannot predict R_i from C_i without physical access (unpredictability). $\square$

Theorem 2 (Replay Attack Resistance). *The protocol prevents replay attacks in all phases.*

Proof: (1) Fresh timestamps T_1, T_2, T_3, T_4 in all messages; messages with $|T_{\text{current}} - T_i| > \Delta T_{\text{max}}$ are rejected. (2) Fresh random nonces N_1, N_2 ensure message uniqueness across sessions. (3) CRPs are marked “consumed” after use—replaying M_3 with a previously used (C_i, R_i) pair fails GS verification. (4) Phase 3 nonces n_i, n_j are ephemeral and bound into MACs η_1, η_2, η_3 . $\square$

Theorem 3 (Peer Mutual Authentication). *Phase 3 provides mutual authentication between UAV_i and UAV_j without GS involvement.*

Proof: (1) UAV_j is authenticated by UAV_i because decrypting puzzle $Q_i = E_{P_{ij}}(n_j \| C_i \| T_{p1})$ requires knowledge of P_{ij} , which requires $P_{ji} = h(R_j \| \text{ID}_i \| N_{\text{gs}})$, derivable only from $R_j = \text{PUF}_j(C_j)$ (unclonability). (2) Symmetrically, UAV_i is authenticated via Q_j . (3) The XOR masks $\text{Mask}_{ij} = P_{ij} \oplus P_{ji}$ are information-theoretically secure one-time pads. $\square$

Theorem 4 (MITM Resistance). *The protocol resists man-in-the-middle attacks.*

Proof: (1) All messages are MAC-protected: $\alpha_1, \beta_1, \gamma_1, \delta_1$ in Phase 2 and η_1, η_2, η_3 in Phase 3. Modification is detected

via MAC verification failure. (2) Encrypted puzzles Q_i, Q_j cannot be decrypted or re-encrypted without PUF-derived secrets P_{ij}, P_{ji} . (3) The PUF response R_i is transmitted only in masked form $(R_i \oplus h(N_2))$, preventing extraction. $\square$

Theorem 5 (Physical Capture Resistance). *Capture of UAV_i does not compromise other UAVs or past sessions.*

Proof: (1) NVM contains only TID_i (rotated after each session), τ_i (useless without PUF to regenerate R_i), and masks Mask_{ij} (require R_j from other PUFs to extract P_{ji}). (2) No raw PUF responses are stored. (3) Invasive probing to extract the PUF physically destroys it (tamper evidence). (4) Session key $\text{SK}_{ij} = h(n_i \| n_j \| n'_i \| R_i \| R_j)$ uses ephemeral nonces; past keys cannot be reconstructed. With optional ECDH, perfect forward secrecy is achieved. $\square$

X. DISCUSSION

Platform considerations: Our x86-64 results provide accurate *relative* comparisons. On ARM Cortex-M4, SHA3 would degrade $2\text{--}20\times$; with FPGA/ASIC acceleration, hash timing becomes negligible. Real 802.11 MAC overhead (1–10 ms/frame) would affect all protocols equally.

Simulation vs. theory: OMNeT++ measures 175 B (Phase 2) + 104 B (Phase 3) vs. theoretical 210 + 196 B, due to simplified credential packaging. Timing impact: <0.05 ms at 6 Mbps.

Limitations: Fixed UAV set (dynamic join/leave needs Phase 1 extension); 12 CRPs limit sessions before re-enrollment; single-GS design needs extension for multi-GS deployments.

XI. CONCLUSION

We presented a four-phase PUF-based authentication protocol for UAV swarm networks, combining PUF hardware root-of-trust with both UAV–GS and direct peer-to-peer authentication in a unified framework. The complete protocol was implemented and evaluated in OMNeT++ 6.3 using a realistic 10-UAV stadium surveillance scenario with dual hash modes. Key findings:

- **Phase 2 (UAV–GS):** 0.672 ms (SHA3) / 7.368 ms (SPONGENT)—10–22 $\times$ faster than RSA/ECC alternatives
- **Phase 3 (peer):** 0.476 ms (SHA3) / 14.535 ms (SPONGENT) per pair, with 100% success across all 45 full-mesh pairs
- **Complete swarm:** 28.1 ms (SHA3) / 727.8 ms (SPONGENT software) for full 10-UAV mutual authentication
- **Overhead:** 279 bytes/pair (simulation); sub-1 KB storage per UAV
- **Hardware:** SPONGENT on FPGA (2,190 GE) achieves 33 ms total swarm authentication—near-identical to SHA3 with $5\times$ less area and power
- **Security:** Mutual authentication, clone resistance, MITM/replay resistance, no key storage, physical capture resistance—all proven via formal theorems

The $2,225\times$ SHA3-to-SPONGENT software performance gap collapses to $2\text{--}5\times$ on hardware, confirming SPONGENT’s

suitability as the optimal hash function for FPGA/ASIC-accelerated UAV authentication. All results were obtained on an x86-64 laptop; real deployment on low-power ARM would be $2\text{--}20\times$ slower in software, while FPGA/ASIC acceleration would be $100\text{--}400\times$ faster for SPONGENT.

Future work includes FPGA prototyping on Xilinx Artix-7, MAVLink 2.0 integration for real UAV deployment, adversarial evaluation (jamming, Sybil attacks), large-scale 50–100 UAV simulation, formal ProVerif/Tamarin verification, and post-quantum security assessment.

ACKNOWLEDGMENT

The author thanks the Indian Institute of Information Technology Guwahati for providing the research infrastructure, the OMNeT++ community for the simulation framework, and the OpenSSL project for the SHA3 implementation.

REFERENCES

- [1] A. Mukherjee *et al.*, “Securing UAV Flying Ad Hoc Wireless Networks: Authentication Development for Robust Communications,” *IEEE Access*, vol. 12, pp. 1–15, 2024.
- [2] P. Gope and B. Sikdar, “A PUF-based lightweight authentication and key agreement protocol for smart UAV networks,” *IET Commun.*, vol. 16, no. 18, pp. 2136–2148, 2022.
- [3] T. Bera, S. Das, and A. K. Das, “Secure and Efficient Peer2Peer Authentication-Attestation Protocol for UAV Networks,” in *Proc. IEEE INFOCOM Workshop*, 2023.
- [4] A. Alkathiri *et al.*, “Review of Physically Unclonable Functions (PUFs): Structures, Models, and Algorithms,” *Frontiers in Electronics*, vol. 4, 2023.
- [5] L. Xu *et al.*, “An Enhanced Event-Based Dynamic Authentication Protocol Leveraging Arbiter PUF for IoT Devices,” *IEEE IoT J.*, vol. 10, no. 18, pp. 16123–16135, 2023.
- [6] NIST, “SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions,” FIPS 202, Aug. 2015.
- [7] A. Bogdanov *et al.*, “SPONGENT: A Lightweight Hash Function,” in *Proc. CHES 2011*, LNCS 6917, pp. 312–325.
- [8] J. Delvaux *et al.*, “Cryptographic Key Generation from PUF Data Using Efficient Fuzzy Extractors,” *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 2025–2038, 2020.
- [9] D. Dolev and A. Yao, “On the Security of Public Key Protocols,” *IEEE Trans. Inf. Theory*, vol. 29, no. 2, pp. 198–208, 1983.
- [10] B. Li *et al.*, “PUF-Based Secure and Efficient Anonymous Authentication Protocol for UAV Towards Cross-Domain Environments,” *IEEE Trans. Veh. Technol.*, vol. 72, no. 9, pp. 12101–12115, 2023.
- [11] S. Khan *et al.*, “A Provably Secure and Lightweight PUF-Based Authentication Scheme for 6G-Enabled UAV Networks,” *IEEE IoT J.*, vol. 11, no. 5, pp. 8532–8543, 2024.
- [12] U. Chatterjee *et al.*, “PUF-based Authentication in IoT against Strong Physical Adversary,” in *Proc. IEEE DATE*, 2022.
- [13] T. Eisenbarth *et al.*, “Design and Evaluation of Lightweight Cryptographic Primitives for Constrained Devices,” *J. Cryptographic Eng.*, vol. 12, no. 3, pp. 245–260, 2022.
- [14] G. Bertoni *et al.*, “The Keccak Reference,” Version 3.0, Jan. 2011.
- [15] R. Maes, “Helper Data Masking for PUF-Based Key Generation,” Cryptology ePrint Archive, Report 2019/688, 2019.
- [16] S. Kumar *et al.*, “Design of A Ring Oscillator Based PUF with Enhanced Challenge Response pair and Improved Reliability,” *IEEE Access*, vol. 11, pp. 75432–75445, 2023.
- [17] L. Zhang *et al.*, “SRAM-PUF Key Extraction Method and Performance Analysis Based on RS and BCH Codes,” *Electronics*, vol. 12, no. 4, p. 1012, 2023.